

Lecture9

面向对象设计模式

主讲老师：高祥



2024/11/6

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

软件设计七大原则

- 单一职责原则

- 一个类只负责一个功能领域中的相应职责。

- 开闭原则

- 一个软件实体应当对扩展开放，对修改关闭。

- 合成复用原则

- 尽量使用组合或者聚合关系实现代码复用，少使用继承。

- 依赖倒转原则

- 抽象不应该依赖于细节，细节应当依赖于抽象。
换言之，要针对接口编程，而不是针对实现编程。

软件设计七大原则

- 接口隔离原则

- 使用多个专门的接口, 而不使用单一的总接口, 即客户端不应该依赖那些它不需要的接口。

- 迪米特法则:

- 一个软件实体应当尽可能少地与其他实体发生相互作用。

- 里氏代换原则

- 所有引用基类（父类）的地方必须能透明地使用其子类的对象。

什么是设计模式？

- 一个设计模式（pattern）是针对某一类问题的最佳解决方案，而且已经被成功应用于许多系统的设计中，它解决了在某种特定情景中重复发生的某个问题，即一个设计模式是从许多优秀的软件系统中总结出的成功的可复用的**设计方案**。
- **学习设计模式的必要性**
 - 通过了解设计模式体现面向对象的设计思想，非常有利于更好地使用面向对象语言解决设计中的诸多问题。

设计模式分类

- 创建型

- 涉及**对象的实例化**，这类模式的特点是，不让用户代码依赖于对象的创建或排列方式，**避免用户直接使用new运算符创建对象**。例如：工厂方法模式，单例模式

- 结构型

- 涉及如何**组合类和对象**以形成更大的结构，和类有关的结构型模式涉及如何**合理地使用继承机制**，和对象有关的结构型模式涉及如何合理地**使用对象组合机制**。例如：装饰模式等

- 行为型

- 涉及怎样合理地设计对象之间的**交互通信**，以及怎样合理为**对象分配职责**，让设计富有弹性，易维护，易复用。例如：策略模式，中介者模式，责任链模式，访问者模式等。

注意事项

- **需求驱动**：设计模式不是为每个人准备的，而是基于**业务来选择设计模式**，需要时就能想到它。要明白一点，技术永远为业务服务，技术只是满足业务需要的一个工具。
- 我们需要掌握每种设计模式的**应用场景、特征、优缺点，以及每种设计模式的关联关系**，这样就能够很好地满足日常业务的需要。

注意事项

- 在进行设计时，尽可能用最简单的方式满足系统的要求，而不是费尽心机地琢磨如何在这个问题中使用设计模式。
- 一个设计可能并不需要使用设计模式就可以很好地满足系统的要求，如果牵强地使用某个设计模式可能会在系统中增加许多额外的类和对象，影响系统的性能。

主要内容 - 重要的设计模式

- 创建型
 - 单例模式
 - 工厂模式（简单工厂、抽象工厂）
 - 原型（Prototype）模式
- 结构型
 - 适配器模式
 - 装饰模式
 - 门面模式
- 行为型
 - 策略模式
 - 访问者模式
 - 责任链模式
 - 观察者模式

创建型模式

- 1. 单例 (Singleton) 模式**: 某个类只能生成一个实例, 该类提供了一个全局访问点供外部获取该实例, 其拓展是有限多例模式。
- 2. 工厂方法 (FactoryMethod) 模式**: 定义一个用于创建产品的接口, 由子类决定生产什么产品。
- 3. 抽象工厂 (AbstractFactory) 模式**: 提供一个创建产品族的接口, 其每个子类可以生产一系列相关的产品。
- 4. 原型 (Prototype) 模式**: 将一个对象作为原型, 通过对其进行复制而克隆出多个和原型类似的新实例。
- 5. 建造者 (Builder) 模式**: 将一个复杂对象分解成多个相对简单的部分, 然后根据不同需要分别创建它们, 最后构建成该复杂对象。

原型 (Prototype) 模式

- 原型 (Prototype) 模式是用一个已经创建的实例作为原型，通过**复制**该原型对象来创建一个和原型相同或相似的新对象。
- 这种模式是实现了一个原型接口，该接口用于创建当前对象的克隆。当直接创建对象的代价比较大时，则采用这种模式。

原型 (Prototype) 模式

- 由于 Java 提供了对象的 clone() 方法，所以用 Java 实现原型模式很简单，只需要实现 Cloneable 接口并重写 clone() 方法，简单程度仅次于单例模式。
- 组成部分
 1. 抽象原型类：规定了具体原型对象必须实现的接口。
 2. 具体原型类：实现抽象原型类的 clone() 方法，它是可被复制的对象。
 3. 访问类：使用具体原型类中的 clone() 方法来复制新的对象。

原型 (Prototype) 模式

```
interface Cloneable{  
    public Object clone() throws CloneNotSupportedException;  
}
```

```
class Realizetype implements Cloneable{  
    Realizetype() {  
        System.out.println("具体原型创建成功！");  
    }  
    public Object clone() throws CloneNotSupportedException {  
        System.out.println("具体原型复制成功！");  
        return (Realizetype)super.clone();  
    }  
}
```

具体原型创建成功！
具体原型复制成功！
obj1==obj2?false

```
public class PrototypeTest{  
    public static void main(String[] args)throws CloneNotSupportedException {  
        Realizetype obj1=new Realizetype();  
        Realizetype obj2=(Realizetype)obj1.clone();  
        System.out.println("obj1==obj2?"+(obj1==obj2));  
    }  
}
```



原型模式优点与使用场景

- 原型模式优点

- 性能优良：原型模式是在内存二进制流的拷贝，要比new一个对象性能好很多，特别是在一个循环体类产生大量对象的时候更加明显。
- 逃避构造函数的约束：这是优缺点共存的一点，直接在内存中拷贝，构造函数是不会执行的。

- 使用场景

- 资源初始化场景：类初始化需要消耗非常多的资源的时候。
- 性能和安全要求的场景：通过new产生一个对象需要非常繁琐的数据准备和访问权限的时候。
- 一个对象多个修改者的场景：一个对象需要提供给其他对象访问，而各个调用者可能都需要修改其值时考虑使用。

结构型模式

- 1. 适配器 (Adapter) 模式:** 将一个类的接口转换成客户希望的另外一个接口, 使得原本由于接口不兼容而不能一起工作的那些类能一起工作。
- 2. 装饰 (Decorator) 模式:** 动态地给对象增加一些职责, 即增加其额外的功能。
- 3. 外观 (Facade) 模式:** 为多个复杂的子系统提供一个一致的接口, 使这些子系统更加容易被访问。
- 4. 代理 (Proxy) 模式:** 为某对象提供一种代理以控制对该对象的访问。即客户端通过代理间接地访问该对象, 从而限制、增强或修改该对象的一些特性。
- 5. 桥接 (Bridge) 模式:** 将抽象与实现分离, 使它们可以独立变化。它使用组合关系代替继承关系来实现的, 从而降低了抽象和实现这两个可变维度的耦合度。
- 6. 享元 (Flyweight) 模式:** 运用共享技术来有效地支持大量细粒度对象的复用。
- 7. 组合 (Composite) 模式:** 将对象组合成树状层次结构, 使用户对单个对象和组合对象具有一致的访问性。

适配器模式

适配器将一个类的接口转换成客户希望的另外一个接口

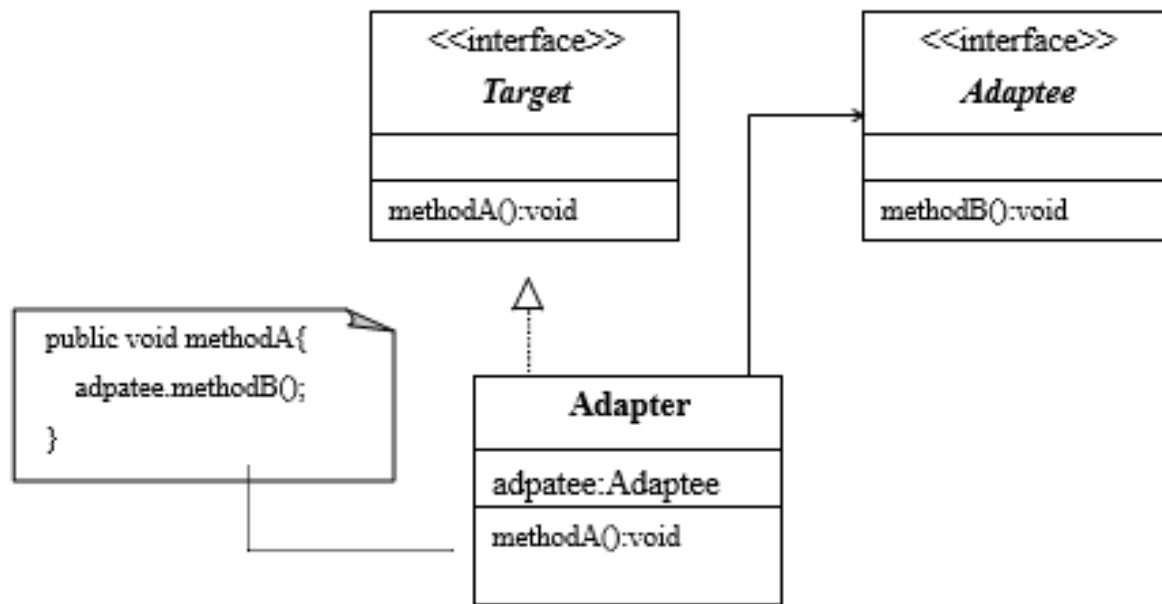


三脚转换头



适配器模式中包括三种角色：

- **目标 (Target)**：目标是一个接口，该接口是客户想使用的接口。
- **被适配者 (Adaptee)**：被适配者是一个已经存在的接口或抽象类，这个接口或抽象类需要适配。
- **适配器 (Adapter)**：适配器是一个类，该类实现了目标接口并包含有被适配者的引用，即适配器的职责是对被适配者接口（抽象类）与目标接口进行适配。



适配器模式案例

- 用户家里现有一台洗衣机，使用交流电，现在用户新买了一台录音机，录音机只能使用直流电。
- 由于供电系统供给用户家里是交流电，因此用户需要用适配器将交流电转化为直流电供录音机使用
 - 目标（Target）：是名字为DirectCurrent的接口
 - 被适配者（Adaptee）：是名字为AlternateCurrent的接口
 - 适配器：是名字为ElectricAdapter类，该类实现了DirectCurrent接口并包含有AlternateCurrent接口变量。
 - 被适配者（Adaptee）的具体类：PowerCompany

被适配者

```
public interface AlternateCurrent{  
    public String giveAlternateCurrent();  
}
```

PowerCompany. java

```
class PowerCompany implements AlternateCurrent { //交流电提供者  
    public String giveAlternateCurrent(){  
        return "10101010101010101010"; //用这样的串表示交流电  
    }  
}
```

目标 (Target)

```
public interface DirectCurrent{  
    public String giveDirectCurrent();  
}
```

Recorder. java

```
class Recorder { //录音机使用直流电  
    String name;  
    Recorder(){  
        name="录音机";  
    }  
    Recorder(String s){  
        name=s;  
    }  
    public void turnOn(DirectCurrent a){  
        String s=a.giveDirectCurrent();  
        System.out.println(name+"使用直流电:\n"+s);  
        System.out.println("开始录音。");  
    }  
}
```

3. 适配器

```
public class ElectricAdapter implements DirectCurrent{
    AlternateCurrent out;
    ElectricAdapter(AlternateCurrent out){
        this.out=out;
    }
    public String giveDirectCurrent(){
        String m = out.giveAlternateCurrent(); //先由out得到交流电
        StringBuffer str =new StringBuffer(m);
        //以下将交流电转为直流电:
        for(int i=0;i<str.length();i++) {
            if(str.charAt(i)=='0') {
                str.setCharAt(i,'1');
            }
        }
        m =new String(str);
        return m; //返回直流电
    }
}
```

模式的使用（洗衣机使用交流电）

```
class Wash { //洗衣机使用交流电
    String name;
    Wash(){
        name="洗衣机";
    }
    Wash(String s){
        name=s;
    }
    public void turnOn(AlternateCurrent a){
        String s=a.giveAlternateCurrent();
        System.out.println(name+"使用交流电:\n"+s);
        System.out.println("开始洗衣物。");
    }
}
```

模式的使用（收音机使用直流电）

```
public class Application{  
    public static void main(String args[]){  
        AlternateCurrent aElectric =new PowerCompany(); //交流电  
        Wash wash =new Wash();  
        wash.turnOn(aElectric); //洗衣机使用交流电  
        //对交流电aElectric进行适配得到直流电dElectric:  
        DirectCurrent dElectric = new ElectricAdapter(aElectric); //将交流电适配成直流电  
        Recorder recorder =new Recorder();  
        recorder.turnOn(dElectric); //录音机使用直流电  
    }  
}
```

洗衣机使用交流电：
10101010101010101010
开始洗衣服。
录音机使用直流电：
11111111111111111111
开始录音

适配器的适配程度

- 1. 完全适配

- 如果目标 (Target) 接口中的方法数目与被适配者 (Adaptee) 接口的方法数目相等, 那么适配器 (Adapter) 可将被适配者接口 (抽象类) 与目标接口进行完全适配。

- 2. 不完全适配

- 如果目标 (Target) 接口中的方法数目少于被适配者 (Adaptee) 接口的方法数目, 那么适配器 (Adapter) 只能将被适配者接口 (抽象类) 与目标接口进行部分适配。

- 3. 剩余适配

- 如果目标 (Target) 接口中的方法数目大于被适配者 (Adaptee) 接口的方法数目, 那么适配器 (Adapter) 可将被适配者接口 (抽象类) 与目标接口进行完全适配, 但必须将目标多余的方法给出用户允许的默认实现。

Practice

- 一位用户拿着业界**标准开关** StandardSwitcher，它依赖 IStandardSwitchable 接口才能工作。然而目前我们有个**灯**并不支持这个接口，如何实现**适配器**让标准开关支持我们的灯。

```
public class StandardSwitcher{  
    IStandardSwitchable device;  
    public StandardSwitcher(IStandardSwitchable device){  
        this.device = device;  
    }  
    public void turnOn(){  
        device.turnOn();  
    }  
    public void turnOff(){  
        device.turnOff();  
    }  
}
```

```
public interface IStandardSwitchable{  
    public void turnOn();  
    public void turnOff();  
}
```

Practice

```
public class StandardSwitcher{
    IStandardSwitchable device;
    public StandardSwitcher(IStandardSwitchable device){
        this.device = device;
    }
    public void turnOn(){
        device.turnOn();
    }
    public void turnOff(){
        device.turnOff();
    }
}
```

```
public class SwitcherAdapter
    implements IStandardSwitchable{
    Light light;
    public SwitcherAdapter(Light light){
        this.light = light;
    }
    public void TurnOn(){
        light.TurnOn();
    }
    public void TurnOff() {
        light.TurnOff();
    }
}
```

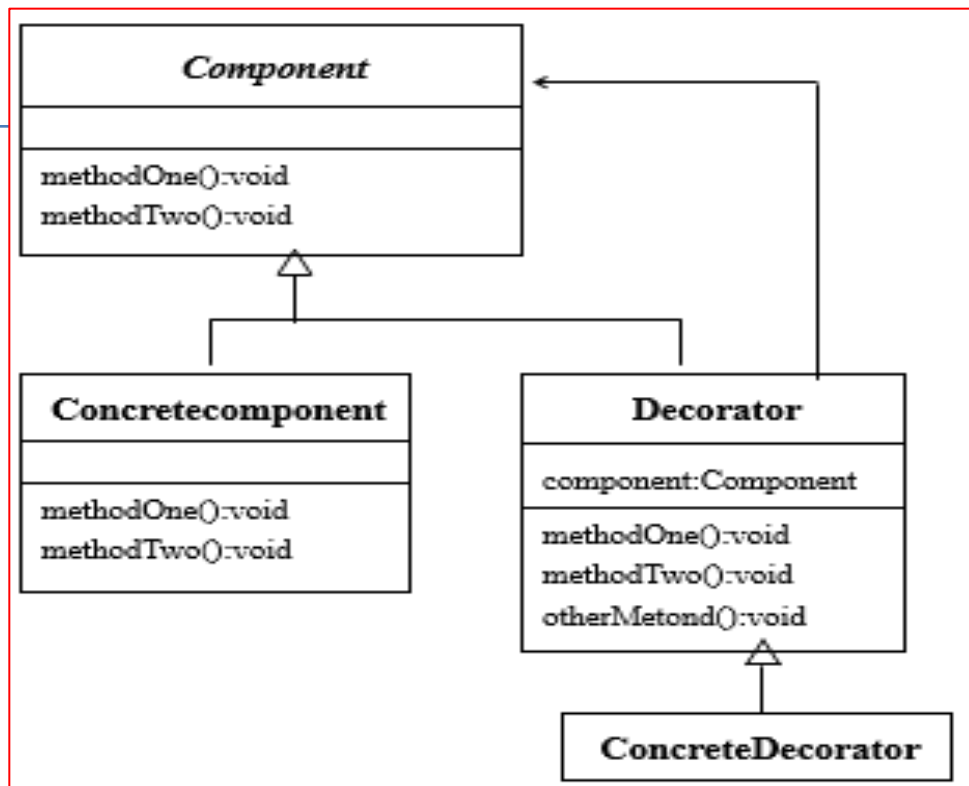
```
public interface IStandardSwitchable{
    public void turnOn();
    public void turnOff();
}
```

```
Light light = new Light();
IStandardSwitchable switchee = new SwitcherAdapter(light);
StandardSwitcher ss = new StandardSwitcher(switchee);
ss.turnOn();
ss.turnoff();
```

“加一层可以解决一切问题”

装饰模式

装饰模式动态地给对象增加一些职责，即增加其额外的功能。



结构中包含以下4种角色：

- **抽象组件（Component）**：抽象组件（是抽象类）定义了需要进行装饰的方法。抽象组件就是“被装饰者”角色。
- **具体组件（ConcreteComponent）**：具体组件是抽象组件的一个子类。
- **装饰（Decorator）**：该角色是抽象组件的一个子类，是“装饰者”角色，其作用是装饰具体组件。Decorator角色需要包含抽象组件的引用。
- **具体装饰（ConcreteDecotator）**：具体装饰是Decorator角色的一个非抽象子类

- 给麻雀安装智能电子翅膀
 - 抽象组件的名字是Bird
 - 装饰（Decorator）角色是抽象组件的一个子类，
需要包含被装饰者（抽象组件）的引用。
 - 具体组件角色：Sparrow类，该类的实例模拟麻雀
 - 具体装饰是SparrowDecorator类，该类使用eleFly()方法去装饰fly()方法

1. 抽象组件

```
Bird.java  
public abstract class Bird{  
    public abstract int fly();  
}
```

2. 具体组件

```
public class Sparrow extends Bird{  
    public final int DISTANCE=100;  
    public int fly(){  
        return DISTANCE;  
    }  
}
```

3. 装饰 (Decorator)

```
public abstract class Decorator extends Bird{  
    Bird bird;    //被装饰者  
    public Decorator(){  
    }  
    public Decorator(Bird bird){  
        this.bird=bird;  
    }  
    public abstract int eleFly();//用于装饰fly()的方法,行为由具体装饰者去实现  
}
```

4. 具体装饰

```
public class SparrowDecorator extends Decorator{
    public final int DISTANCE=50; //eleFly方法(模拟电子翅膀)能飞50米
    SparrowDecorator(Bird bird){
        super(bird);
    }
    public int fly(){ //被装饰的方法
        int distance=0;
        distance=bird.fly()+eleFly();//让装饰者bird首先调用fly(), 然后再调用eleFly()
        return distance;
    }
    public int eleFly(){ //具体装饰者重写装饰者中用于装饰的方法
        return DISTANCE;
    }
}
```

模式的使用:

```
public class Application{
    public static void main(String args[]){
        Bird bird=new Sparrow();
        System.out.println("没有安装电子翅膀的小鸟飞行距离:"+bird.fly());
        bird=new SparrowDecorator(bird); //bird通过"装饰"安装了1个电子翅膀
        System.out.println("安装1个电子翅膀的小鸟飞行距离:"+bird.fly());
        bird=new SparrowDecorator(bird); //bird通过"装饰"安装了2个电子翅膀
        System.out.println("安装2个电子翅膀的小鸟飞行距离:"+bird.fly());
        bird=new SparrowDecorator(bird); //bird通过"装饰"安装了3个电子翅膀
        System.out.println("安装3个电子翅膀的小鸟飞行距离:"+bird.fly());
    }
}
```

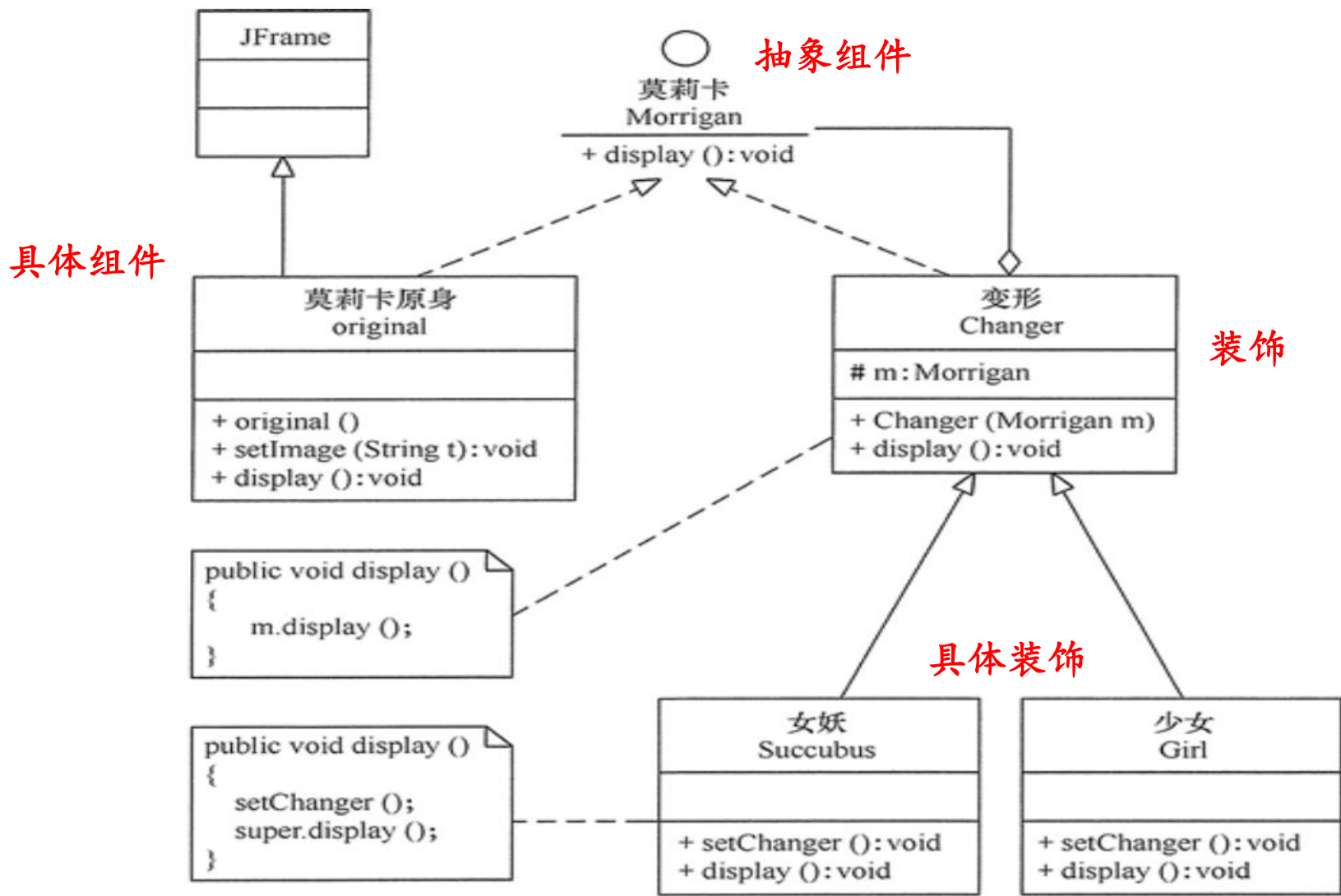
没有安装电子翅膀的小鸟的飞行距离: 100
安装1个电子翅膀的小鸟的飞行距离: 150
安装2个电子翅膀的小鸟的飞行距离: 200
安装3个电子翅膀的小鸟的飞行距离: 250

演示了一只没有安装电子翅膀的小鸟只能飞行100米, 对该鸟进行“装饰”, 即给它安装一个电子翅膀, 那么安装了1个电子翅膀后的鸟就能飞行150米, 然后在继续给它安装电子翅膀, 那么安装了2个电子翅膀后的鸟就能飞行200米

practice



Practice



相对继承机制的优势

- 通过继承也可改进对象的行为，对于某些简单的问题这样做未尝不可，但是如果考虑到系统扩展性，就应当注意面向对象的一个基本原则之一：**少用继承，多用组合**。就功能来说装饰模式相比生成子类更为灵活。
- 如果继续采用继承机制来维护上面的系统，就必须修改系统，不断增加新的Bird的子类，这简直是维护的一场灾难。

模式的优点

- 被装饰者和装饰者是松耦合关系。由于装饰（Decorator）仅仅依赖于抽象组件（Component），因此具体装饰只知道它要装饰的对象是抽象组件的某一个子类的实例，但不需要知道是哪一个具体子类装饰模式，相比生成子类更为灵活。

适合的情景

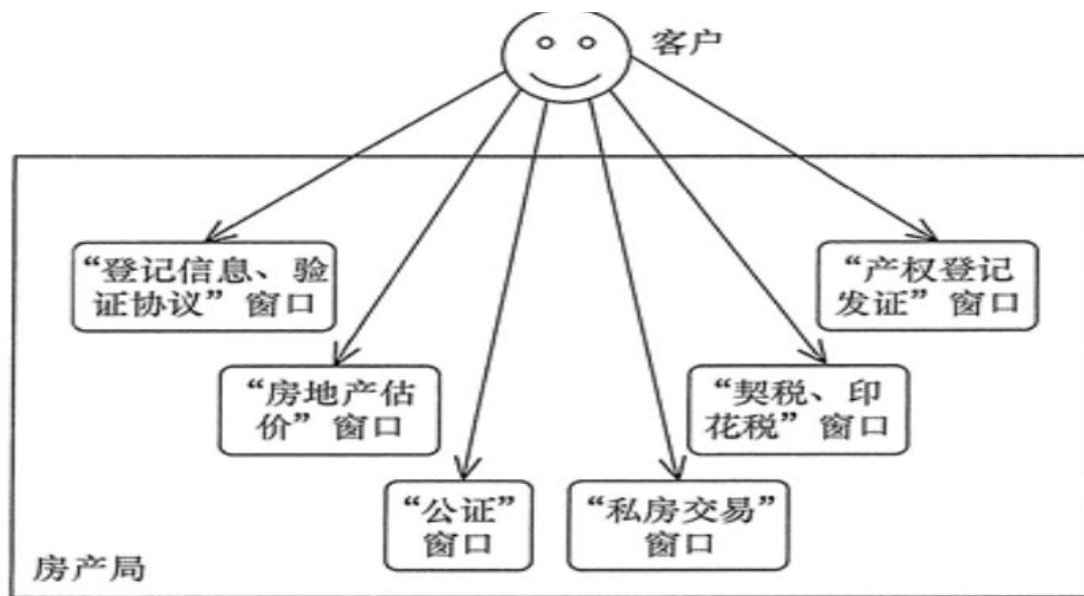
- 程序希望动态地增强类的某个对象的功能，而又不影响到该类的其他对象。
- 采用继承来增强对象功能不利于系统的扩展和维护。

外观模式

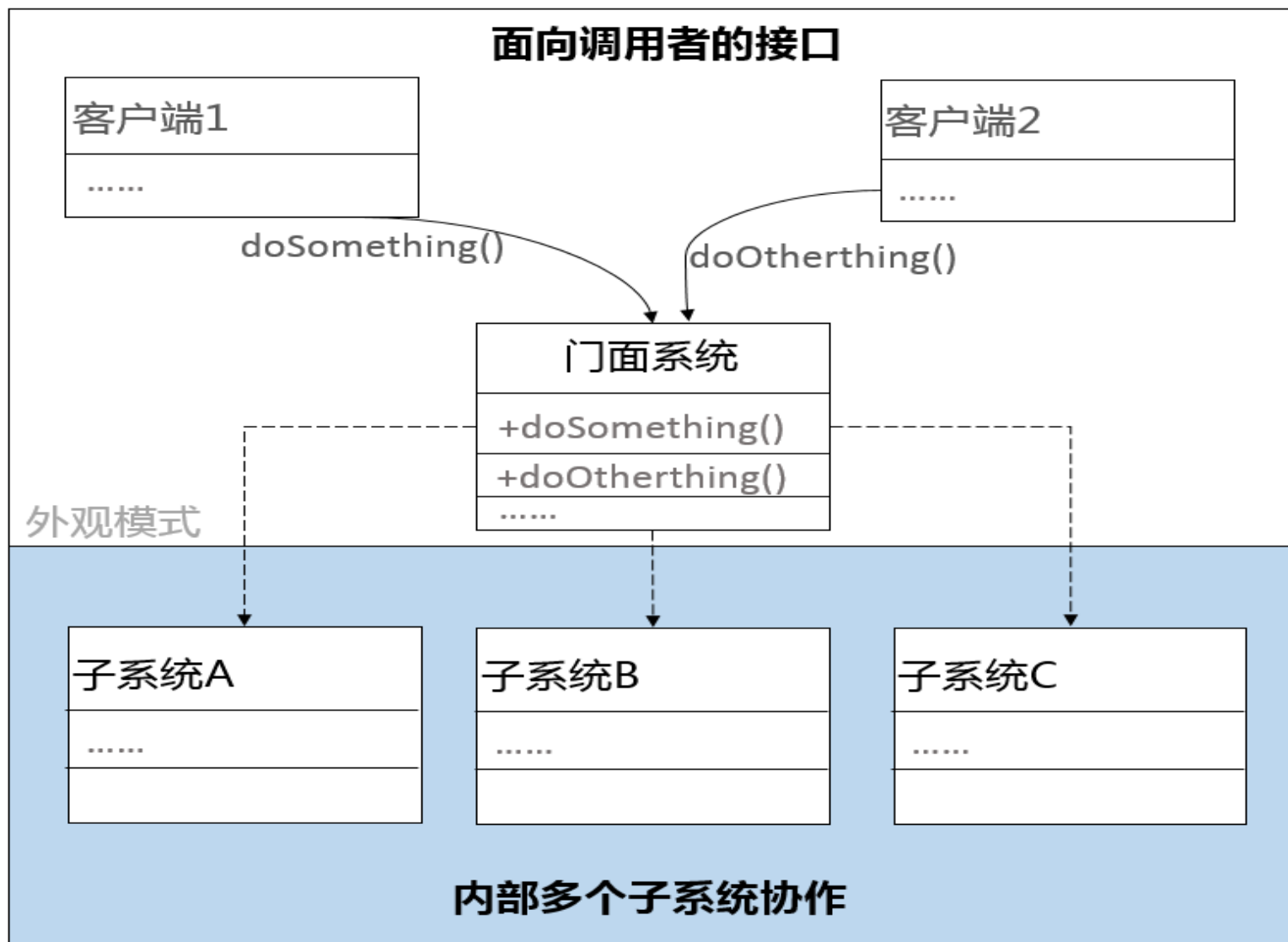
- 外观（Facade）模式又叫作门面模式，是一种通过为多个复杂的子系统提供一个一致的接口，而使这些子系统更加容易被访问的模式。
- 该模式对外有一个统一接口，外部应用程序不用关心内部子系统的具体细节，这样会大大降低应用程序的复杂度，降低其与子系统的耦合，提高了程序的可维护性。
- 外观（Facade）模式是“迪米特法则”的典型应用

外观模式实例

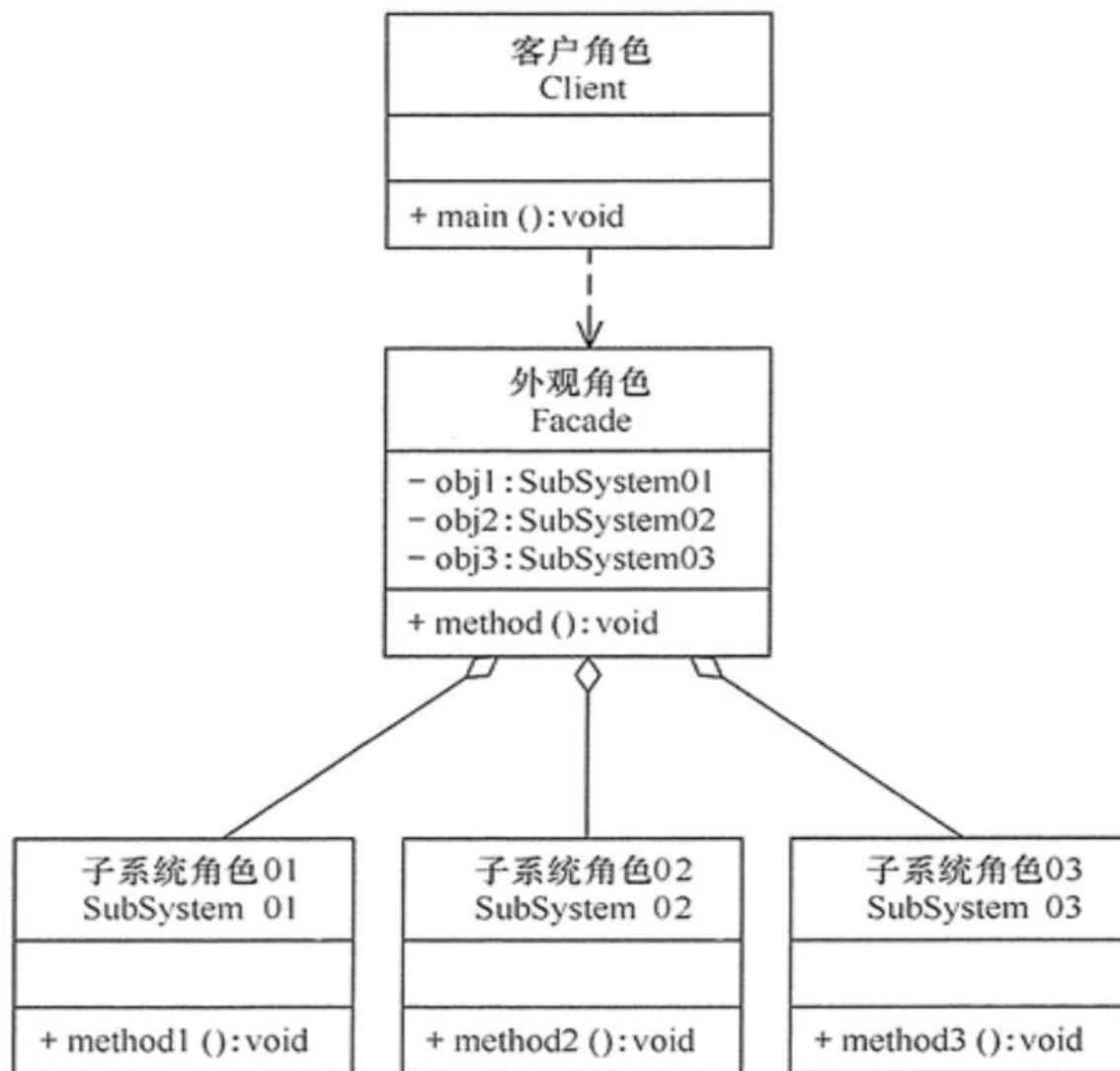
- 在现实生活中，常常存在办事较复杂的例子，如办房产证或注册一家公司，有时要同多个部门联系，这时要是有一个综合部门能解决一切手续问题就好了。
- 客户去当地房产局办理房产证过户要遇到的相关部门



外观 (Facade) 模式的结构图



案例：外观（Facade）模式的结构图



示例代码:

```
//子系统角色
class SubSystem01 {
    public void method1() {
        System.out.println("子系统01的method1()被调用!");
    }
}

//子系统角色
class SubSystem02 {
    public void method2() {
        System.out.println("子系统02的method2()被调用!");
    }
}

//子系统角色
class SubSystem03 {
    public void method3() {
        System.out.println("子系统03的method3()被调用!");
    }
}
```



```
public class FacadePattern {  
    public static void main(String[] args) {  
        Facade f = new Facade();  
        f.method();  
    }  
}  
  
//外观角色  
class Facade {  
    private SubSystem01 obj1 = new SubSystem01();  
    private SubSystem02 obj2 = new SubSystem02();  
    private SubSystem03 obj3 = new SubSystem03();  
  
    public void method() {  
        obj1.method1();  
        obj2.method2();  
        obj3.method3();  
    }  
}
```

外观 (Facade) 模式的优点

外观 (Facade) 模式是“迪米特法则”的典型应用，它有以下主要优点。

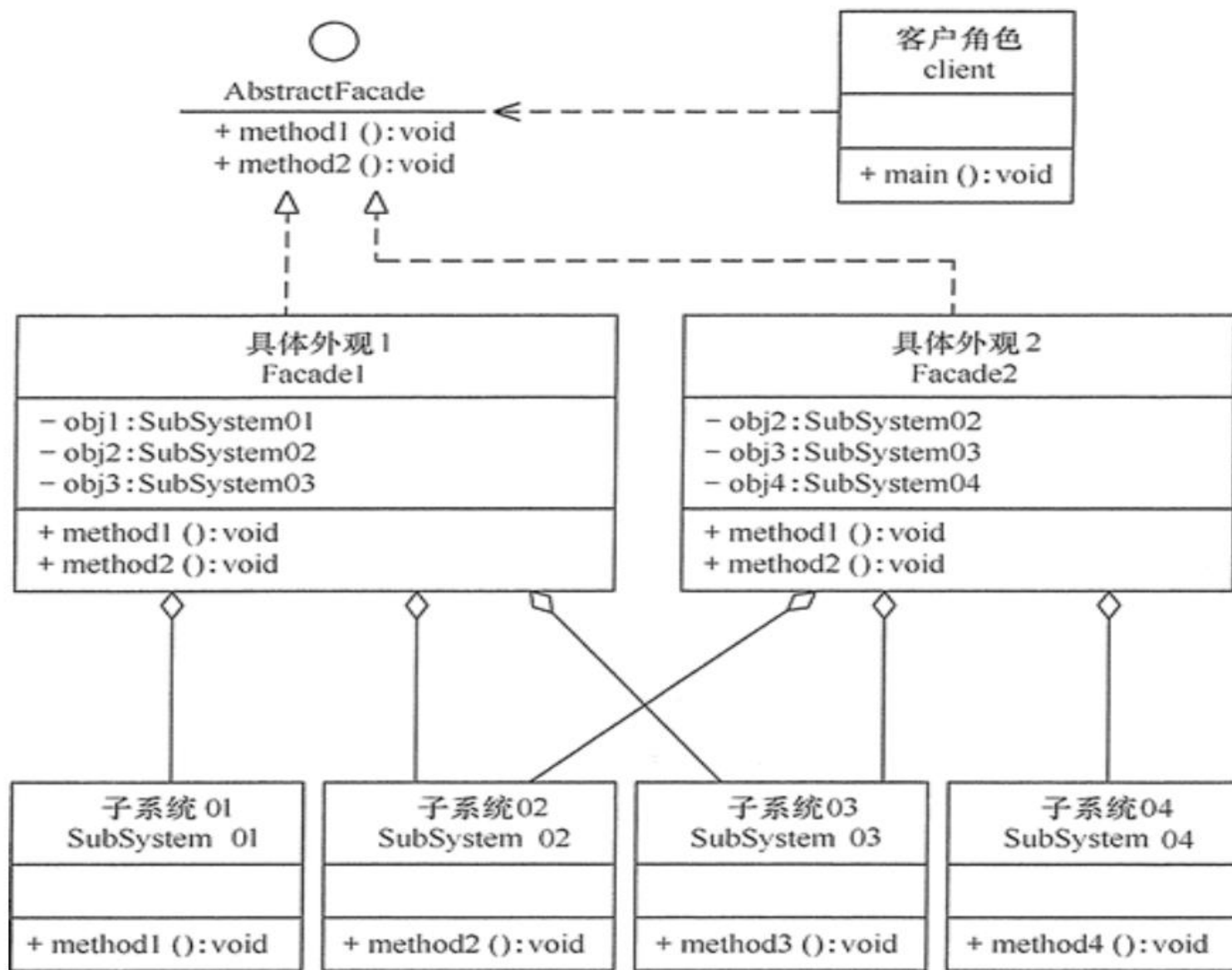
1. 降低了子系统与客户端之间的耦合度，使得子系统的变化不会影响到调用它的客户类。
2. 对客户屏蔽了子系统组件，减少了客户处理的对象数目，并使得子系统使用起来更加容易。
3. 降低了大型软件系统中的编译依赖性，简化了系统在不同平台之间的移植过程，因为编译一个子系统不会影响到其他的子系统，也不会影响到外观对象。

外观 (Facade) 模式的主要缺点

外观 (Facade) 模式的主要缺点如下：

- 1、不能很好地限制客户使用子系统类，很容易带来未知风险。
- 2、增加新的子系统可能需要修改外观类或客户端的源代码，**违背了“开闭原则”**。

外观模式的扩展



外观模式的扩展

- 在外观模式中，当增加或移除子系统时需要修改外观类，这违背了“开闭原则”。
- 如果引入抽象外观类，则在一定程度上解决了该问题。

结构型模式

- 1. 代理 (Proxy) 模式:** 为某对象提供一种代理以控制对该对象的访问。即客户端通过代理间接地访问该对象，从而限制、增强或修改该对象的一些特性。
- 2. 适配器 (Adapter) 模式:** 将一个类的接口转换成客户希望的另外一个接口，使得原本由于接口不兼容而不能一起工作的那些类能一起工作。
- 3. 装饰 (Decorator) 模式:** 动态地给对象增加一些职责，即增加其额外的功能。
- 4. 外观 (Facade) 模式:** 为多个复杂的子系统提供一个一致的接口，使这些子系统更加容易被访问。
- 5. 桥接 (Bridge) 模式:** 将抽象与实现分离，使它们可以独立变化。它是用组合关系代替继承关系来实现的，从而降低了抽象和实现这两个可变维度的耦合度。
- 6. 享元 (Flyweight) 模式:** 运用共享技术来有效地支持大量细粒度对象的复用。
- 7. 组合 (Composite) 模式:** 将对象组合成树状层次结构，使用户对单个对象和组合对象具有一致的访问性。

行为型模式

- 1. 策略 (Strategy) 模式:** 定义了一系列算法，并将每个算法封装起来，使它们可以相互替换，且算法的改变不会影响使用算法的客户。
- 2. 访问者 (Visitor) 模式:** 在不改变集合元素的前提下，为一个集合中的每个元素提供多种访问方式，即每个元素有多个访问者对象访问。
- 3. 职责链 (Chain of Responsibility) 模式:** 把请求从链中的一个对象传到下一个对象，直到请求被响应为止。通过这种方式去除对象之间的耦合。
- 4. 观察者 (Observer) 模式:** 多个对象间存在一对多关系，当一个对象发生改变时，把这种改变通知给其他多个对象，从而影响其他对象的行为。
- 5. 迭代器 (Iterator) 模式:** 提供一种方法来顺序访问聚合对象中的一系列数据，而不暴露聚合对象的内部表示。
- 6. 命令 (Command) 模式:** 将一个请求封装为一个对象，使发出请求的责任和执行请求的责任分割开。

行为型模式

7. **状态 (State) 模式**: 允许一个对象在其内部状态发生改变时改变其行为能力。
8. **中介者 (Mediator) 模式**: 定义一个中介对象来简化原有对象之间的交互关系, 降低系统中对象间的耦合度, 使原有对象之间不必相互了解。
9. **模板方法 (Template Method) 模式**: 定义一个操作中的算法骨架, 将算法的一些步骤延迟到子类中, 使得子类在可以不改变该算法结构的情况下重定义该算法的某些特定步骤。
10. **备忘录 (Memento) 模式**: 在不破坏封装性的前提下, 获取并保存一个对象的内部状态, 以便以后恢复它。
11. **解释器 (Interpreter) 模式**: 提供如何定义语言的文法, 以及对语言句子的解释方法, 即解释器。

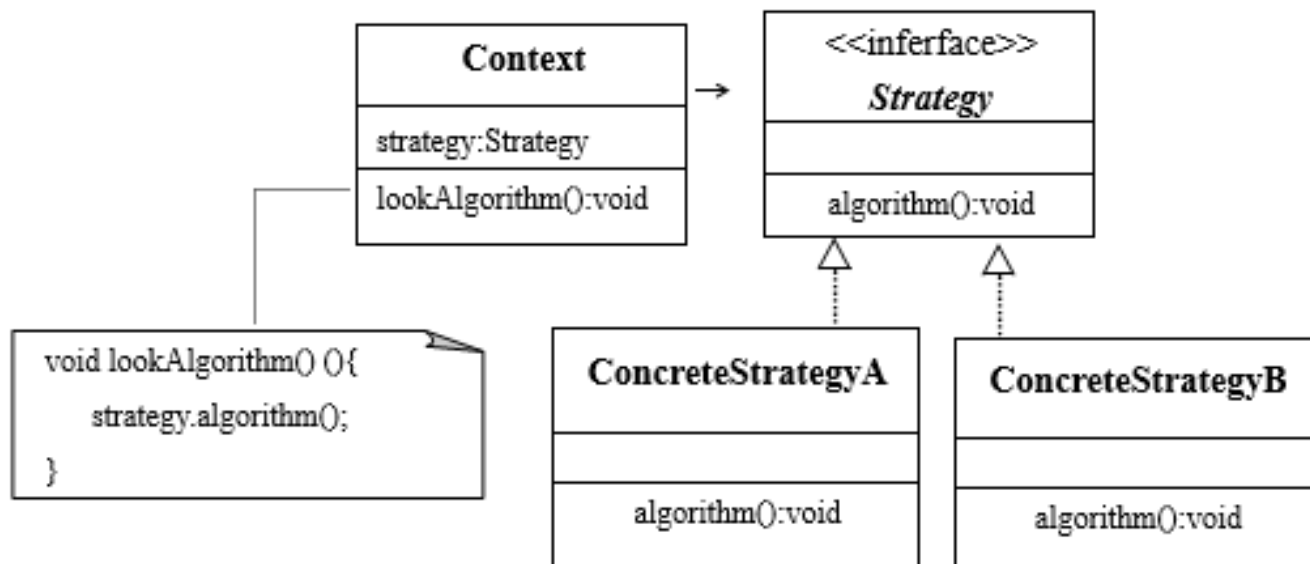
策略模式

策略（Strategy）模式的核心就是将类中经常需要变化的部分分割出来，并将每种可能的变化对应地交给抽象类的一个子类或实现接口的一个类去负责，从而让类的设计者不去关心具体实现，避免所设计的类依赖于具体的实现。

策略模式

结构中包含以下三种角色：

- **策略 (Strategy)**：策略是一个接口，该接口定义若干个算法标识，即定义了若干个抽象方法。
- **具体策略 (ConcreteStrategy)**：具体策略是实现策略接口的类。具体策略实现策略接口所定义的抽象方法，即给出算法标识的具体算法。
- **上下文 (Context)**：上下文是依赖于策略接口的类（是面向策略设计的类），即上下文包含有用策略声明的变量。上下文中提供一个方法，该方法委托策略变量调用具体策略所实现的策略接口中的方法。



- 问题：在多个裁判负责打分的比赛中，每位裁判给选手一个得分，选手的最后得分是根据全体裁判的得分计算出来的。请给出几种计算选手得分的评分方案（策略），对于某次比赛，可以从你的方案中选择一种方案作为本次比赛的评分方案。
 - 在这里我们把策略接口命名为：**Strategy**。在具体应用中，这个角色的名字可以根据具体问题来命名。
 - 在本问题中将上下文命名为**AverageScore**，即让AverageScore类依赖于Strategy接口。
 - 每个具体策略负责一系列算法中的一个。

1. 策略 (Strategy) Strategy.java

```
public interface Strategy {  
    public double computerAverage(double [] a);  
}
```

2. 上下文 (Context) AverageScore.java

```
public class AverageScore{  
    Strategy strategy;  
    public void setStrategy(Strategy strategy){  
        this.strategy=strategy;  
    }  
    public double getAverage (double [] a){  
        if(strategy!=null)  
            return strategy.computerAverage(a);  
        else {  
            System.out.println("没有求平均值的算法,得到的-1不代表平均值");  
            return -1;  
        }  
    }  
}
```

3. 具体策略StrategyA.java

```
public class StrategyA implements Strategy{  
    public double computerAverage(double [] a){  
        double score=0,sum=0;  
        for(int i=0;i<a.length;i++){  
            sum=sum+a[i];  
        }  
        score=sum/a.length;  
        return score;  
    }  
}
```

4. 具体策略StrategyB.java

```
import java.util.Arrays;
public class StrategyB implements Strategy{
    public double computerAverage(double [] a){
        if(a.length<=2)
            return 0;
        double score=0,sum=0;
        Arrays.sort(a); //排序数组
        for(int i=1;i<a.length-1;i++){
            sum=sum+a[i];
        }
        score=sum/(a.length-2);
        return score;
    }
}
```

5. 模式的使用

```
class Person{
    String name;
    double score;
    public void setScore(double t){
        score=t;
    }
    public void setName(String s){
        name=s;
    }
    public double getScore(){
        return score;
    }
    public String getName(){
        return name;
    }
}
```

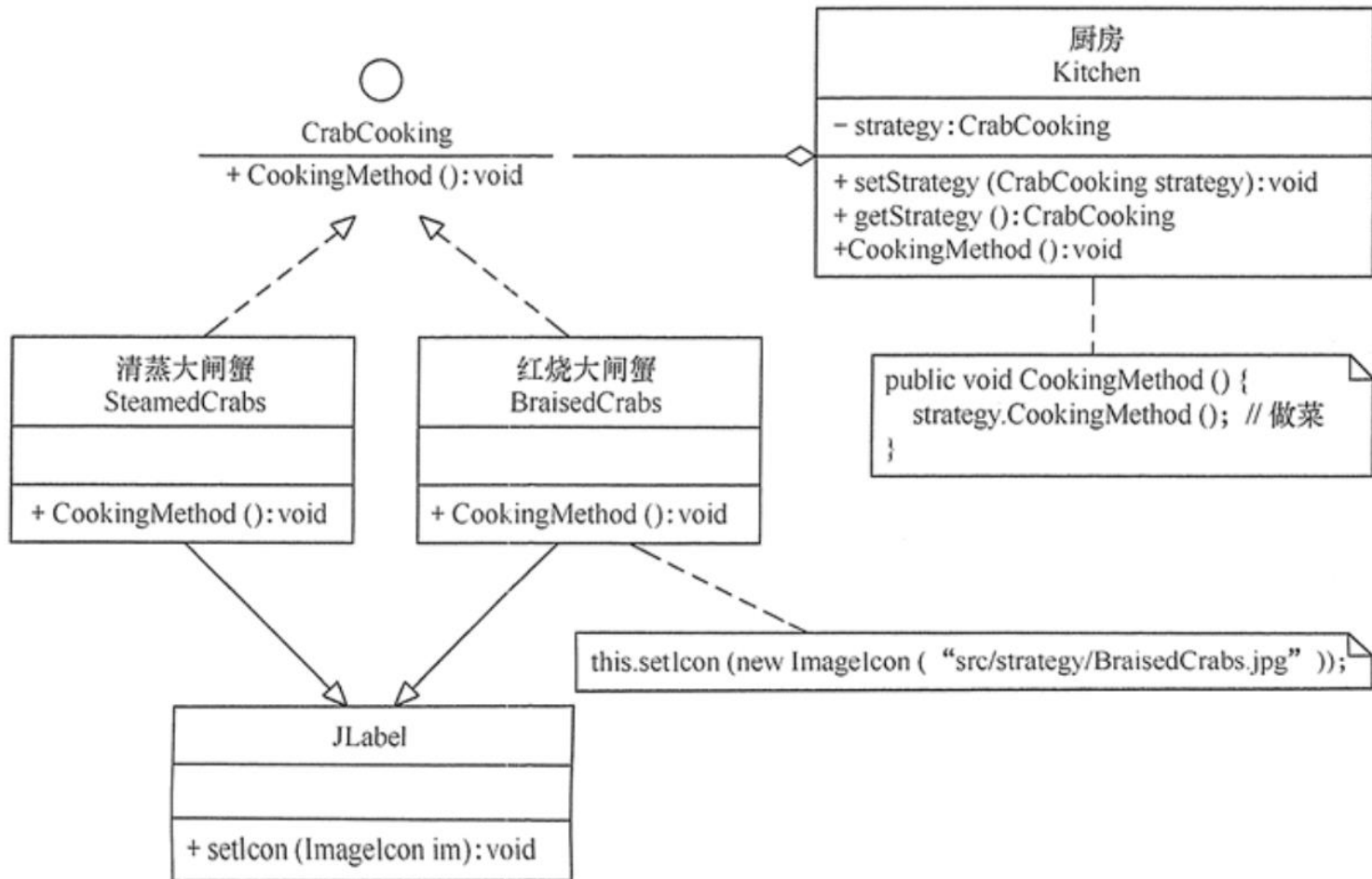
5. 模式的使用

```
public class Application{
    public static void main(String args[]){
        AverageScore game=new AverageScore();//上下文对象game
        game.setStrategy(new StrategyA()); //上下文对象使用具体策略
        Person zhang=new Person();
        zhang.setName("张三");
        double [] a={9.12,9.25,8.87,9.99,6.99,7.88};
        double aver = game.getAverage(a); //上下文对象得到
        zhang.setScore(aver);
        System.out.println("算法A:");
        System.out.printf("%s最后得分:%5.3f%n",zhang.getName(),zhang.getScore());
        game.setStrategy(new StrategyB());
        aver = game.getAverage(a); //上下文对象得到平均值
        zhang.setScore(aver);
        System.out.println("算法B:");
        System.out.printf("%s最后得分:%5.3f%n",zhang.getName(),zhang.getScore());
    }
}
```

算法A:
张三最后得分: 8.683
算法B:
张三最后得分: 8.780

Practice

家里买了两只大闸蟹，你想吃红烧的，你妈妈想吃清蒸的，想一想这个问题如何使用策略模式来实现？想一想策略（Strategy），具体策略和上下文（Context）分别是什么？



模式的优点

- 上下文（Context）和具体策略（ConcreteStrategy）是松耦合关系。因此上下文只知道它要使用某一个实现Strategy接口类的实例，但不需要知道具体是哪一个类。
- 策略模式满足“开-闭原则”。当增加新的具体策略时，不需要修改上下文类的代码，上下文就可以引用新的具体策略的实例。

适合的场景

- 程序的主要类（相当于上下文角色）不希望暴露复杂的、与算法相关的数据结构，那么可以使用策略模式封装算法，即将算法分别封装到具体策略中

相对继承机制的优势

- 考虑到系统扩展性和复用性，就应当注意面向对象的一个基本原则之一：**少用继承，多用组合**。
- 策略模式的应用层次采用的是组合结构，即将上下文类的某个方法的内容的不同变体分别封装在不同的具体策略中。
- 如果将父类的某个方法的内容的不同变体交给对应的子类去实现，就使得这些实现和父类中的其它代码是**紧耦合关系**，因为父类的任何改动都会影响到子类。

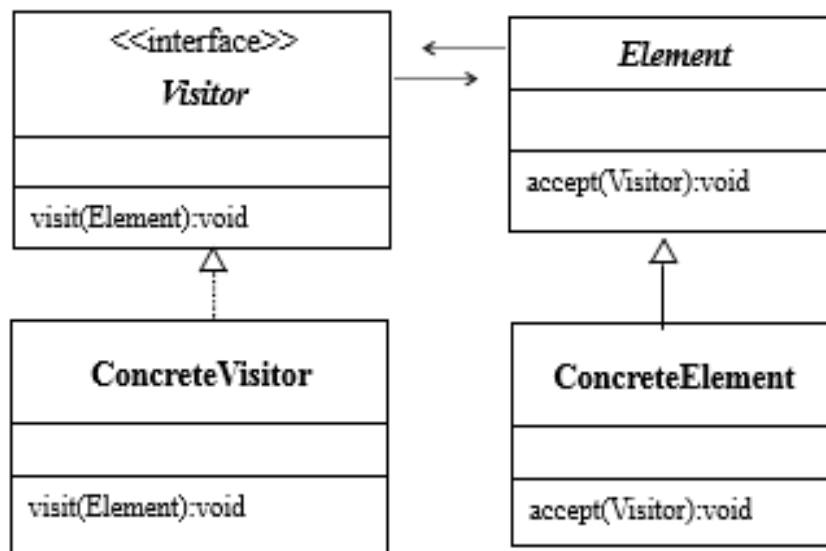
访问者模式模式

- 某个类可能用自己的实例方法操作自己的数据，但在某些设计中，可能需要定义**作用于类中的数据的新操作**，而且这个**新的操作不应当由该类中的某个实例方法来承担**。
 - 比如，电表有自己的显示用电量的方法（用显示盘显示），但需要定义一个方法来计算电费，即需要定义作用于电量的新操作，显然这个新的操作不应当由电表来承担。
 - 在实际生活中，应当由物业部门的“计表员”观察电表的用电量，然后按着有关收费标准计算出电费。**让一个称作访问者的对象访问电表**，并根据用电量来计算电费。

访问者模式

结构中包含以下4种角色：

- 抽象元素（Element）：一个抽象类，该类定义了接收访问者的accept操作。
- 具体元素（Concrete Element）：Element的子类。
- 抽象访问者（Visitor）：一个接口，该接口定义操作具体元素的方法。
- 具体访问者（Concrete Visitor）：实现Visitor接口的类。



- 问题：根据电表显示的用电量计算用户的电费。用户包括居民和企业。访问同一个电表，即分别按家用电标准和工业用电标准计算了电费。
 - 抽象的访问者：
 - 具体的访问者：居民和企业用户
 - 抽象元素：抽象类AmmeterElement
 - 具体元素：Ammeter(模拟电表)

1. 抽象访问者 (Visitor)

```
public interface Visitor{  
    public double visit(AmmeterElement element);  
}
```

2. 抽象元素 (Element)

```
public abstract class AmmeterElement{  
    public abstract void accept(Visitor v);  
    public abstract double showElectricAmount();  
    public abstract void setElectricAmount(double n);  
}
```


3. 具体访问者 (Concrete Visitor)

```
public class HomeAmmeterVisitor implements Visitor{  
    public double visit(AmmeterElement ammeter){  
        double charge=0;  
        double unitOne=0.6,unitTwo=1.05;  
        int basic = 6000;  
        double n=ammeter.showElectricAmount();  
        if(n<=basic) {  
            charge = n*unitOne;  
        }  
        else {  
            charge =basic*unitOne+(n-basic)*unitTwo;  
        }  
        return charge;  
    }  
}
```

3. 具体访问者 (Concrete Visitor)

```
public class IndustryAmmeteVisitor implements Visitor{
    public double visit(AmmeterElement ammeter){
        double charge=0;
        double unitOne=1.52,unitTwo=2.78;
        int basic = 15000;
        double n=ammeter.showElectricAmount();
        if(n<=basic) {
            charge = n*unitOne;
        }
        else {
            charge =basic*unitOne+(n-basic)*unitTwo;
        }
        return charge;
    }
}
```



4. 具体元素 (Concrete Element)

```
public class Ammeter extends AmmeterElement{
    double electricAmount;  //电表的电量
    public void setElectricAmount(double n) {
        electricAmount = n;
    }
    public void accept(Visitor visitor){
        double feiyong=visitor.visit(this); //让访问者访问当前元素
        System.out.println("当前电表的用户需要交纳电费:"+feiyong+"元");
    }
    public double showElectricAmount(){
        return electricAmount;
    }
}
```

模式的使用:

```
public class Application{  
    public static void main(String args[]) {  
        Visitor 计表员=new HomeAmmeterVisitor(); //按家用电表标准计算电费的"计表员"  
        Ammeter 电表=new Ammeter();  
        电表.setElectricAmount(5678);  
        电表.accept(计表员);  
        计表员=new IndustryAmmeteVisitor(); //按工业用电标准计算电费的"计表员"  
        电表.setElectricAmount(5678);  
        电表.accept(计表员);  
    }  
}
```

当前电表的用户需要缴纳电费3406.79999元
当前电表的用户需要缴纳电费8630.56元

Practice:

年底，CEO和CTO开始评定员工一年的工作绩效，员工为工程师，CTO关注工程师的代码量；CEO关注的是工程师的KPI。如何使用访问者模式？抽象访问者，抽象元素，具体访问者分别指什么？

```
// 员工基类
public abstract class Staff {

    public String name;
    public int kpi;// 员工KPI

    public Staff(String name) {
        this.name = name;
        kpi = new Random().nextInt(10);
    }
    // 核心方法，接受Visitor的访问
    public abstract void accept(Visitor visitor);
}
```

```
// 工程师
public class Engineer extends Staff {
    public Engineer(String name) {
        super(name);
    }
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
    // 工程师一年的代码数量
    public int getCodeLines() {
        return new Random().nextInt(10 * 1000);
    }
}
```

Practice:

// 工程师

```
public class Engineer extends Staff {  
    public Engineer(String name) {  
        super(name);  
    }  
    public void accept(Visitor visitor) {  
        visitor.visit(this);  
    }  
    // 工程师一年的代码数量  
    public int getCodeLines() {  
        return new Random().nextInt(10 * 1000);  
    }  
}
```

```
public interface Visitor {  
    void visit(Engineer engineer);  
}
```

// CEO访问者

```
public class CEOVisitor implements Visitor  
{  
    public int visit(Engineer en) {  
        System.out.println("Name: " +  
en.name + ", KPI: " + en.kpi);  
    }  
}
```

// CTO访问者

```
public class CTOVisitor implements Visitor  
{  
    public void visit(Engineer en) {  
        System.out.println("Name: " +  
en.name + ", KPI: " + en.getCodeLines());  
    }  
}
```

模式优点和使用场景

- **模式优点：**在不改变一个集合中的元素的类的情况下，可以增加新的施加于该元素上的新操作。保持一定的扩展性。
- **使用场景：**需要对集合中的对象进行很多不同的并且不相关的操作，而我们又不想修改对象的类，就可以使用访问者模式。访问者模式可以在Visitor类中集中定义一些关于集合中对象的操作。

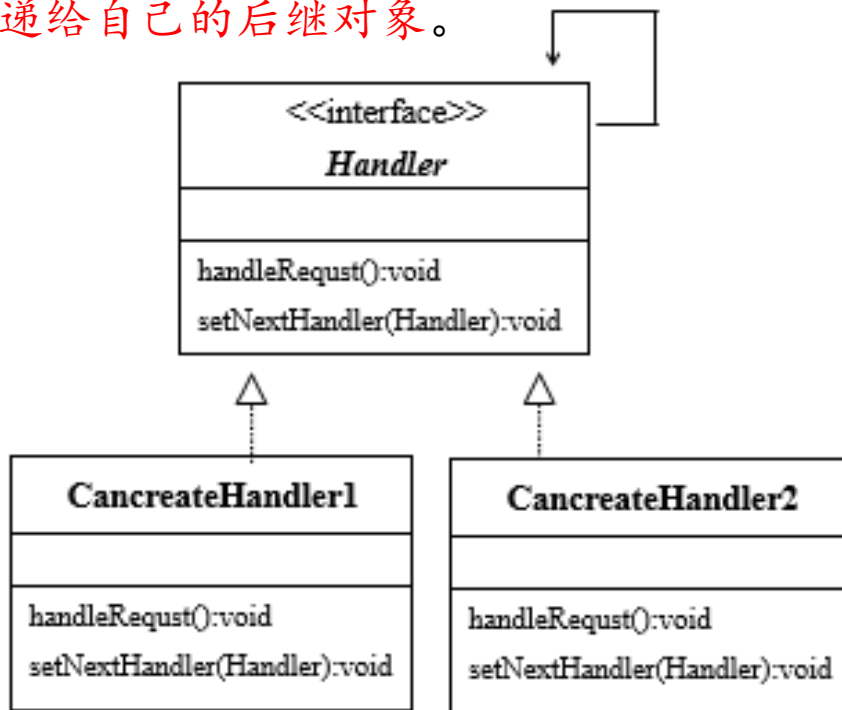
责任链模式

责任链模式是使用多个对象处理用户请求的成熟模式,责任链模式的关键是**将用户的请求分派给许多对象**

责任链模式

结构中包含以下2种角色：

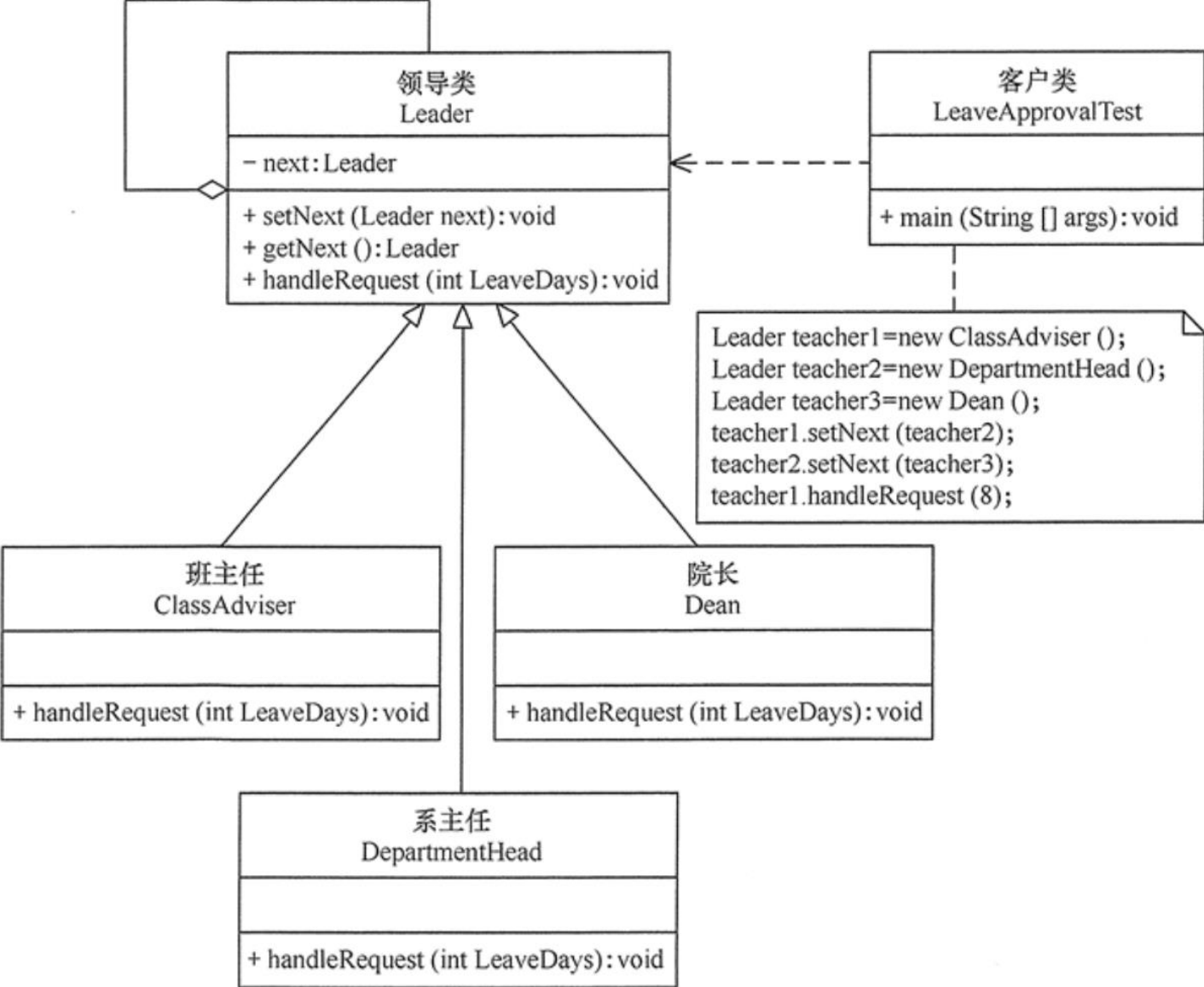
- **处理者 (Handler)**：处理者是一个接口，负责规定具体处理者**处理用户的请求的方法**以及具体处理者设置**后继对象**的方法。
- **具体处理者 (ConcreteHandler)**：具体处理者是**实现处理者接口**的类的实例。具体处理者通过调用处理者接口规定的方法处理用户的请求，即在接到用户的请求后，处理者将调用接口规定的方法，在执行该方法的过程中，如果发现能处理用户的请求，就处理有关数据，否则就反馈无法处理的信息给用户，然后将**用户的请求传递给自己的后继对象**。



案例：用责任链模式设计一个请假条审批模块

- 假如规定学生请假小于或等于 2 天，班主任可以批准；小于或等于 7 天，系主任可以批准；小于或等于 10 天，院长可以批准；其他情况不予批准；这个实例适合使用职责链模式实现。

案例：请假条审批模块的结构图



1、Leader.java

//抽象处理者：领导类

```
abstract class Leader {  
    private Leader next;
```

```
    public void setNext(Leader next) {  
        this.next = next;  
    }
```

```
    public Leader getNext() {  
        return next;  
    }
```

//处理请求的方法

```
    public abstract void handleRequest(int LeaveDays);  
}
```

2、ClassAdviser.java

//具体处理者1：班主任类

```
class ClassAdviser extends Leader {  
    public void handleRequest(int LeaveDays) {  
        if (LeaveDays <= 2) {  
            System.out.println("班主任批准您请假" + LeaveDays + "天。");  
        } else {  
            if (getNext() != null) {  
                getNext().handleRequest(LeaveDays);  
            } else {  
                System.out.println("请假天数太多，没有人批准该假条！");  
            }  
        }  
    }  
}
```

3、DepartmentHead.java

//具体处理者2：系主任类

```
class DepartmentHead extends Leader {  
    public void handleRequest(int LeaveDays) {  
        if (LeaveDays <= 7) {  
            System.out.println("系主任批准您请假" + LeaveDays + "天。");  
        } else {  
            if (getNext() != null) {  
                getNext().handleRequest(LeaveDays);  
            } else {  
                System.out.println("请假天数太多，没有人批准该假条！");  
            }  
        }  
    }  
}
```

4、Dean.java

//具体处理者3：院长类

```
class Dean extends Leader {  
    public void handleRequest(int LeaveDays) {  
        if (LeaveDays <= 10) {  
            System.out.println("院长批准您请假" + LeaveDays + "天。");  
        } else {  
            if (getNext() != null) {  
                getNext().handleRequest(LeaveDays);  
            } else {  
                System.out.println("请假天数太多，没有人批准该假条！");  
            }  
        }  
    }  
}
```

测试类

```
public class LeaveApprovalTest {  
    public static void main(String[] args) {  
        //组装责任链  
        Leader teacher1 = new ClassAdviser();  
        Leader teacher2 = new DepartmentHead();  
        Leader teacher3 = new Dean();  
        //Leader teacher4=new DeanOfStudies();  
        teacher1.setNext(teacher2);  
        teacher2.setNext(teacher3);  
        //teacher3.setNext(teacher4);  
        //提交请求  
        teacher1.handleRequest(8);  
    }  
}
```

程序运行结果如下:

院长批准您请假8天。

假如增加一个教务处长类，可以批准学生请假 20 天，也非常简单

//具体处理者4: 教务处长类

```
class DeanOfStudies extends Leader {  
    public void handleRequest(int LeaveDays) {  
        if (LeaveDays <= 20) {  
            System.out.println("教务处长批准您请假" + LeaveDays + "天。");  
        } else {  
            if (getNext() != null) {  
                getNext().handleRequest(LeaveDays);  
            } else {  
                System.out.println("请假天数太多，没有人批准该假条！");  
            }  
        }  
    }  
}
```

责任链模式

- 模式的优点:

- 当在处理者中分配职责时,责任链给应用程序更多的灵活性。

- 使用场景

- 有许多对象可以处理用户的请求。
 - 程序希望动态制定可处理用户请求的对象集合

Practice

- 假如电影票7元, 购票者购买2张电影票, 给售票员100元面值钞票一张。那么实际上售票员找赎86元过程如下:
 - 首先在50元面值的钱盒中看能否完成任务, 或部分任务。50元面值的钱盒发现能找赎86元中的50元(贡献一张50元的钞票), 即只完成部分任务, 因此把剩余的36元任务交给下一个20元面值的钱盒
 - 20元面值的钱盒发现能完成36元中的20元(贡献一张20面值的钞票), 因此把剩余的16元任务交给下一个10元面值的钱盒
 - 依次类推, 如果在后续某个钱盒完成了自己的找赎任务, 那么售票员就完成了找赎任务, 如果一直到最后一个钱盒(假设是1元面值的钱盒), 该钱盒也无法完成自己的找赎任务, 那么售票员就无法完成找赎
 - 使用责任链模拟售票员找赎, 那么责任链上的对象就是钱盒, 即责任链模式中的处理者(Handler)

Practice

- 问题中,责任链上的处理者接口的名字是 **MoneyHandler**,负责规定具体处理者使用哪些方法来处理用户的请求以及规定具体处理者设置后继对象的方法。
- 具体处理者就是实现处理着接口 **MoneyHandler**的类,**不同面值的钱**
- **TickerSeller**将使用框架中的 **MoneyBox**创建责任链,并使用该责任链进行找赎。

观察者（Observer）模式（监听模式）

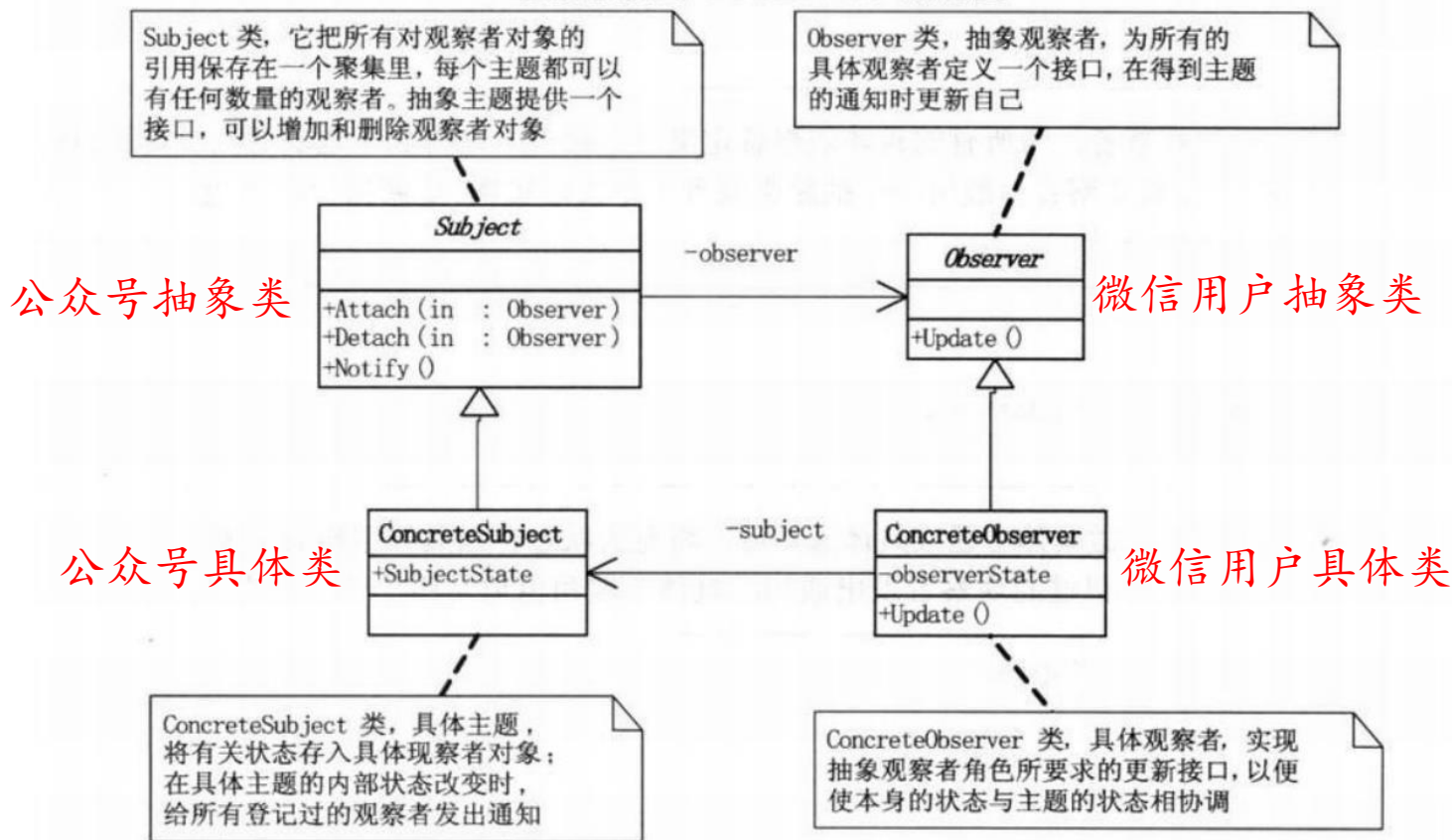
多个对象间存在一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。这种模式有时又称作发布-订阅模式、模型-视图模式。

1. **抽象主题（Subject）角色**：也叫抽象目标类（**被观察者**），它提供了一个用于保存观察者对象的聚集类和**增加、删除观察者**对象的方法，以及**通知所有观察者**的抽象方法。
2. **具体主题（Concrete Subject）角色**：也叫具体目标类，它实现抽象目标中的**通知方法**，当具体主题的内部状态发生改变时，通知所有注册过的观察者对象。
3. **抽象观察者（Observer）角色**：它是一个抽象类或接口，它包含了一个**更新自己的抽象方法**，当接到具体主题的更改通知时被调用。
4. **具体观察者（Concrete Observer）角色**：实现抽象观察者中定义的抽象方法，以便在得到目标的更改通知时更新自身的状态。

观察者模式

观察者模式定义了一种一对多的依赖关系，让多个观察者对象同时监听某一个主题对象。这个主题对象在状态发生变化时，会通知所有观察者对象，使它们能够自动更新自己。[DP]

观察者模式 (Observer) 结构图



微信订阅公众号

```
public interface Subject {  
    public void attach(Observer observer); //增加订阅者  
    public void detach(Observer observer); //删除订阅者  
    public void notify(String message); //通知订阅者更新消息  
}
```

```
public class SubscriptionSubject implements Subject {  
    private List<Observer> weixinUserlist = new ArrayList<Observer>();  
    public void attach(Observer observer) {  
        weixinUserlist.add(observer);  
    }  
    public void detach(Observer observer) {  
        weixinUserlist.remove(observer);  
    }  
    public void notify(String message) {  
        for (Observer observer : weixinUserlist)  
            observer.update(message);  
    }  
}
```

储存订阅公众号的微信用户

通知所有订阅公众号的微信用户

```
public interface Observer {  
    public void update(String message);  
}
```

```
public class WeixinUser implements Observer {  
    private String name;  
    public WeixinUser(String name) {  
        this.name = name;  
    }  
    public void update(String message) {  
        System.out.println(name + "-" + message);  
    }  
}
```

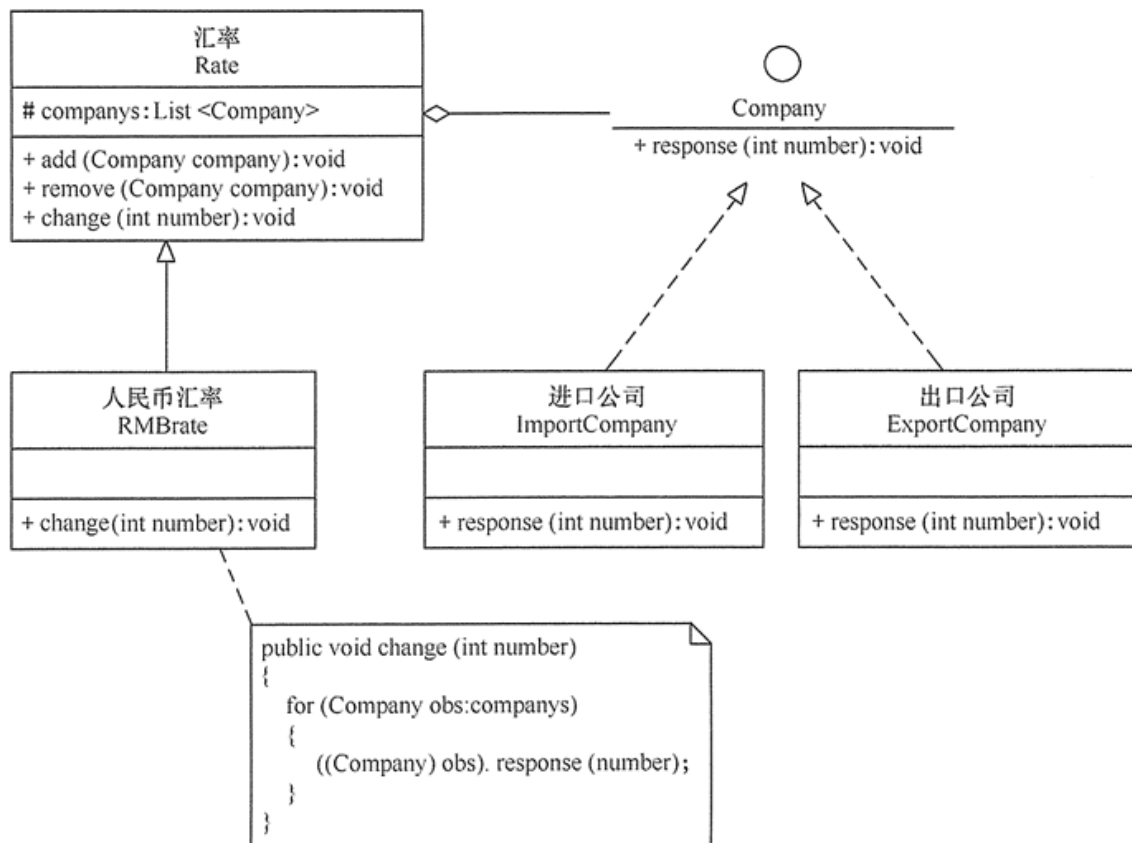

微信订阅公众号

```
public class Client {  
    public static void main(String[] args) {  
        SubscriptionSubject mSubscriptionSubject=new SubscriptionSubject();  
        //创建微信用户  
        WeixinUser user1=new WeixinUser("杨影枫");  
        WeixinUser user2=new WeixinUser("月眉儿");  
        WeixinUser user3=new WeixinUser("紫轩");  
        //订阅公众号  
        mSubscriptionSubject.attach(user1);  
        mSubscriptionSubject.attach(user2);  
        mSubscriptionSubject.attach(user3);  
        //公众号更新发出消息给订阅的微信用户  
        mSubscriptionSubject.notify("量子位公众号更新了");  
    }  
}
```

杨影枫 - 量子位公众号更新了
月眉儿 - 量子位公众号更新了
紫轩 - 量子位公众号更新了

Practice

- 利用观察者模式设计一个程序，分析“人民币汇率”的升值或贬值对进口公司的进口产品成本或出口公司的出口产品收入以及公司的利润率的影响。



使用场景

- 一个抽象模型有两个方面，其中一个方面依赖于另一个方面。将这些方面封装在独立的对象中使它们可以各自独立地改变和复用。
- 一个对象的改变将导致其他一个或多个对象也发生改变，而不知道具体有多少对象将发生改变，可以降低对象之间的耦合度。
- 一个对象必须通知其他对象，而并不知道这些对象是谁。
- 需要在系统中创建一个触发链，A对象的行为将影响B对象，B对象的行为将影响C对象……，可以使用观察者模式创建一种链式触发机制

- 优点:

- 观察者和被观察者是抽象耦合的
- 建立一套触发机制

- 缺点:

- 如果一个被观察者对象有很多的直接和间接的观察者的话，将所有的观察者都通知到会花费很多时间。
- 如果在观察者和观察目标之间有循环依赖的话，观察目标会触发它们之间进行循环调用，可能导致系统崩溃。

小结

1. **原型 (Prototype) 模式** 用一个已经创建的实例作为原型, 通过**复制**该原型对象来创建一个和原型相同或相似的新对象。
2. **适配器模式**: 的核心是将一个类的接口转换成客户希望的另外一个接口。
3. **装饰模式**: 的核心是动态地给对象添加一些额外的职责。
4. **外观模式**: 为多个复杂的子系统提供一个一致的接口, 使这些子系统更加容易被访问
5. **策略模式**: 定义一系列算法, 把它们封装起来, 并且使它们可相互替换。本模式使得算法可独立于使用它的客户而变化。
6. **访问者模式**: 的核心是在不改变各个元素的类的前提下定义作用于这些元素的新操作。
7. **责任链模式**: 把请求从链中的一个对象传到下一个对象, 直到请求被响应为止。通过这种方式去除对象之间的耦合。
8. **观察者模式**: 一对多关系, 当一个对象发生改变时, 把这种改变通知给其他多个对象, 从而影响其他对象的行为。